

um-game-playing-strength-of-mcts-variants

UM - Game-Playing Strength of MCTS Variants 13位解法

Rank	13
Team	G&G
Author	Gabor Balazs
Topic ID	549781
Version	Japanese Translation

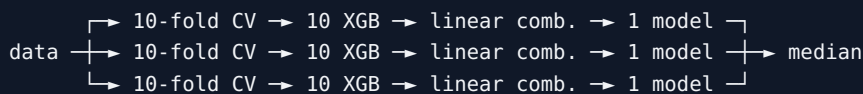
Source: <https://www.kaggle.com/competitions/um-game-playing-strength-of-mcts-variants/discussion/549781>
Retrieved with Kaggle CLI on 2026-07-03.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

素晴らしいコンペ体験を与えてくださった主催者と参加者の皆さんに感謝します。LB 0.420のモデルが多数ある中で、PB 0.425となる唯一のモデルを選べたため、13位になったのは予想外でかなり幸運でした（選んだもの以外はすべてPB 0.426）。

アプローチ概要

最終モデルは、10-fold CV由来の10個のXGBoost (XGB) regressorを線形結合したensembleを3組作り、その中央値を取りました。



agent-flip augmentationと特徴量エンジニアリングも使い、bagging、column subsampling、boostingのearly stoppingで過学習を抑えました。全XGB予測をtarget範囲 $[-1, 1]$ へclipしました。

提出の詳細

null特徴量と分散0の特徴量をすべて落とし、別特徴量との相関が95%以上の列も順番に落としました。

Agent-flip augmentation

他の解法と同様、学習データの `agent1` と `agent2` を交換し、`AdvantageP1` を `1-train['AdvantageP1']` に、`target utility_agent1` の符号を反転しました（最終的に `num_wins_agent1` と `num_losses_agent1` は不使用）。当初と最終モデル学習では、「非現実的すぎる」状況を作ることを恐れ、対称ゲーム（`train['Asymmetric'] == 0`）だけに拡張しました。後に全データでも試しましたが、XGB線形結合は0.422まででした（こちらの調整量はかなり少なめです）。最終候補の第2モデル（LGB 10個とXGB 10個のensemble）は全拡張データで学習し、Public 0.421からPrivate 0.428になりました。

特徴量エンジニアリング

agentのゲーム体験を区別する唯一の特徴量は `AdvantageP1` でした。ほかのrandom-play挙動指標について [Ludiiのコード](#) を読むと、agentに対して対称でした。ゲーム暗記を防ぐためboostingでは強いcolumn subsamplingを使ったため、`AdvantageP1` が高確率で選ばれるよう、gain重要度の高い特徴量と組み合わせた次の派生を追加しました。

```
X = X.with_columns([
    pl.Series('SgnAdvP1MulGameTreeComplexity',
              (X['AdvantageP1']-0.5) * X['GameTreeComplexity']).clip(-300, 300),
    pl.Series('SgnAdvP1MulBalance', (X['AdvantageP1']-0.5) * X['Balance']),
    pl.Series('SgnAdvP1DivVariance1',
              (X['AdvantageP1']-0.5) / (2.0-X['OutcomeUniformity'].clip(0,1))),
    pl.Series('SgnAdvP1DivDrawFrequency',
              (X['AdvantageP1']-0.5) / (1.0+X['DrawFrequency'].clip(0,1))),
    pl.Series('SgnAdvP1MulCompletion', ((X['AdvantageP1']-0.5) * X['Completion'])),
```

```

pl.Series('SgnAdvP1PerPlayoutsPerSeconds',
          (X['AdvantageP1']-0.5) / (X['PlayoutsPerSecond'] + 1e-4)).clip(-0.5, 0.5),
pl.Series('SgnAdvP1DivDurationTurnsNotTimeouts',
          (X['AdvantageP1']-0.5) / (X['DurationTurnsNotTimeouts']+1e-6)).clip(-0.5, 0.5),
pl.Series('SgnAdvP1MulBranchingFactorMedian',
          (X['AdvantageP1']-0.5) * X['BranchingFactorMedian']).clip(-90, 90),
pl.Series('SgnAdvP1MulBranchingFactorAverage',
          (X['AdvantageP1']-0.5) * X['BranchingFactorAverage']).clip(-100, 100),
])

```

盤面の物理的特徴を主に表す多数の特徴量は、gain重要度で比較的上位（50～100位）に出ましたが、汎化するには信じにくかったため削除しました。

```

X = X.drop([
    'PieceNumberAverage', 'PieceNumberMaximum', 'NumPlayableSitesOnBoard'
    'NumColumns', 'NumRows', 'NumCorners', 'NumOuterSites', 'NumLayers',
    'NumVertices', 'NumTopSites', 'NumRightSites', 'NumCentreSites',
    'NumConvexCorners', 'NumConcaveCorners', 'NumPhasesBoard',
    'NumContainers', 'NumComponentsType', 'NumStartComponentsBoard',
    'NumStartComponentsHand', 'NumStartComponents',
])

```

削除によって情報をすべて失わないよう、次も追加しました。

```

X = X.with_columns([
    pl.Series('IsSquaredBoard', X['NumRows'] == X['NumColumns']),
    pl.Series('PlayoutsPerMoves',
              X['PlayoutsPerSecond'] / (X['MovesPerSecond']+1e-6)).clip(0, 1),
    pl.Series('PieceNumberRatio', X['PieceNumberAverage'] / X['PieceNumberMaximum']),
    pl.Series('AverageTurns', X['NumPlayableSitesOnBoard'] / X['PieceNumberAverage']),
    pl.Series('ActionsPerTurn', X['DurationActions'] / X['DurationTurns']).clip(1, 300),
])

```

最後に `GameRulesetName`、`EnglishRules`、`LudRules` も削除しました。残念ながら活用できませんでした。

Boosting

10-fold CVには `GameRulesetName` でgroup化した `StratifiedGroupKFold` を使い、層化クラスは `(y_data*10).round(decimals=0).astype(int)` としました。3つのXGB regressorのパラメータは似ており、違いを配列で示します。

```

max_leaves=[250, 200, 250], max_depth=[35, 30, 35],
n_estimators=[3500, 4000, 3500], learning_rate=[0.04, 0.03, 0.04],
subsample=[0.9, 0.75, 0.9], colsample_bylevel=0.5, colsample_bynode=0.25,
min_child_weight=25, min_split_loss=1e-4, reg_lambda=2.0, reg_alpha=4.0,
max_bin=64, max_cat_threshold=32, grow_policy='lossguide', enable_categorical=True,

```

leaf数とdepthが大きいため、各splitで特徴量を $0.5 * 0.25 = 0.125$ だけ残すcolumn subsamplingで過学習を抑えました。正則化パラメータは初期grid search由来で、終盤には上3行のパラメータだけを調整しました。第1モデルでは学習データの10%をearly stopping（minimum delta $1e-5$ ）に使い、ほかの2モデルでは使いませんでした。各CV学習は同じdata shuffle seed・異なるestimator seedで3～5回行い、splitごとに最良モデルを選んで「CV score」を最適化しました。LBでも効いたためrandom restart手法として正当化しました。

線形結合 (stacking)

CV由来の10 XGBを、全sample上のRMSEを最小化する重みで線形結合しました。clipも考慮するため全モデル・全sampleの予測を `yhats` (10行×sample数) へ入れ、target `y` に対して次を最小化しました。

```
from scipy.optimize import minimize
def fobj(w):
    return (np.mean(np.square(np.clip(np.dot(w, yhats), -1.0, 1.0) - y))
            + 1e5 * min(0, np.min(w))**2)
w = np.ones(yhats.shape[0]) / yhats.shape[0]
res = minimize(fobj, w)
wopt = res.x
rmse_opt = calc_rmse(np.dot(wopt, yhats).clip(-1, 1), y)
```

等重みから探索を始め、負重みをpenaltyし、clipを目的関数へ含めました。最終モデルの3線形結合は次のとおりです。

```
wopt1 = [0.086247, 0.144699, 0.133306, 0.101278, 0.130589,
          0.136385, 0.076667, 0.139516, 0.108264, 0.106763]
wopt2 = [0.1059 , 0.14333 , 0.113467, 0.111721, 0.120991,
          0.137219, 0.097818, 0.12802 , 0.094427, 0.12176 ]
wopt3 = [0.115637, 0.109509, 0.119757, 0.126103, 0.134074,
          0.137645, 0.076969, 0.118834, 0.125846, 0.098289]
```

合計はそれぞれ 1.164、1.175、1.163 で、他の参加者と同様、結果的に予測値を拡大しています。過学習が怖く、非線形結合は検討しませんでした。

その他

LightGBMとCatBoost

LGBを多用してパラメータを調整し、その設定をXGBとCatBoostへ応用しました。CPU上のLGBはほかより2倍以上高速で、GPU quotaを有望なrunへ温存できました。LGBとCatBoostはLB 0.421、XGBは0.420まで改善しました。最終ensemble内の各XGB結合がすべて0.420だったため、個別0.421・結合Public 0.420・Private 0.426のLGB-XGB-CatBoost案ではなくXGB案を選びました。

失敗した試み

- [9位解法](#)と同様、推移律でも拡張しました。対称ゲームだけで、複数反復し矛盾のあるゲームを除外しましたが、CVを改善せずLBは0.002悪化しました。
- 少数のゲーム特性からnearest-neighbor特徴量を追加しましたが、過学習してCVが悪化しました。
- 線形結合の重み合計が1より大きく、LBで一貫して有利だったため、このbiasがPrivateにないことを恐れしました。対称・非対称ゲームで別モデルを学習して弱めようとしたのですが、LB約0.428と大幅に悪く断念しました。
- 予測値を学習データに現れた値へ丸めるとCV誤差が増えました。
- agent-flipしたテスト予測の符号を戻して通常予測と平均する対称化予測は、対称ゲームだけでもLBを0.001悪化させました。

- LudRules で学習時に見たsampleかを推論時判別し、既知ならより攻めたモデルを使う案もLBを改善しました。主催者によればtrain/testでgameの重複はなかったため、当然の結果です。

補足Q&A

yunsuxiaozi: アンサンブルでmedianを使ったのはなぜですか？

Gabor Balazs: 全sample上でmeanとmedianのRMSEを測り、medianがわずかに良かったためです。差は小さく、median 0.2005、mean 0.2011でした。過学習を恐れ、weighted averageは検討しませんでした。

Mart Preusse: XGBへカテゴリ特徴量をどう与えましたか。default処理ですか、encodingしましたか。agentは分割しましたか？

Gabor Balazs: pandas DataFrameのカテゴリ列をそのまま与え、`enable_categorical=True` にしました。鍵は `grow_policy='lossguide'` で、defaultのdepth-wiseはかなり悪かった記憶があります。agentは `selection`、`exploration_const`、`playout`、`score_bounds` に分割してカテゴリ化しました。一意値が8以下の全列もカテゴリ化しました。元の未分割agent特徴量もordinal encodingで残すと良かったため、feature importanceで最も弱かった `score_bounds` 部分だけ除いて残し、過学習への頑健性を高めました。